BAIL security

Token Mill V2
SOL (RUST)

FINAL REPORT

May '2025

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:
Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Tokenmill V2
Website	tokenmill.xyz
Language	Rust
Methods	Manual Analysis
Github repository	https://github.com/traderjoe-xyz/sol-token-mill-v2/commit/b6f98c36d9e2a7c38be4c865f202c87318a55723
Resolution 1	https://github.com/traderjoe-xyz/sol-token-mill-v2/tree/16dd57d33d7635a2d6dee3a3b9978f654de0c317

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed Resolution
High	1	1			
Medium	2	2			
Low	5	2		3	
Informational	6	3		3	
Governance					
Total	14	8		6	

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium-level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless, the issue should be fixed immediately.
Informational	Effects are small and do not pose an immediate danger to the project or users.
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior.

3. Detection

Tokenmill V2

The Token Mill V2 program is a decentralized exchange (DEX) on Solana that facilitates token swaps and market-making with advanced pricing mechanisms.

It allows the creation of token markets with customizable parameters, supporting concentrated liquidity, configurable fees, and multiple swap types with slippage protection.

The program features a robust fee management system, administrative controls, and security measures, all of which are built using the Anchor framework with comprehensive testing and documentation.

Core Invariants:

INV 1: fee_reserve can only be changed if fee_recipient_change_cooldown has passed since the last fee_reserve update

INV 2: market creation must be permissionless

INV 3: market default parameters and config settings can only be changed by the creator

INV 4: During deployment, only token0 is added as liquidity

INV 5: Both liquidites must be immutable

INV 6: sqrt_price_x96 must never go below sqrt_price_a_x96

Appendix: Swapping method

The current system implements 2 IX, which can be used to swap tokens. The swap_with_price_limit IX allows either buying or selling with a maximum or minimum price that the user is willing to swap at. The swap IX provides for four different types of swaps:

- BuyExactIn
- BuyExactOut
- SellExactIn

- SellExactOut

Each of these allows the user to set one exact value (In/Out) and a maximum and minimum value on the other side to control slippage.

Appendix: 2-Phase system

The pool uses a 2-phase system. In the first phase, the price transitions from sqrt_price_a_x96 to sqrt_price_b_x96. After the price reaches it, the trade occurs between sqrt_price_b_x96 and u128::MAX. This allows a graduation of

Appendix: Swapping Math

The swapping math is inspired by the math used in the UniV3 protocol. The math can be broken down into four key functions, which are used to calculate the correct price changes and token deltas:

`get_amount_0()`

$$amount0 = liquidity * \frac{priceA - priceB}{priceA * priceB}$$

`get_amount_1()`

$$amount1 = liquidity * (priceB - priceA)$$

`get_next_sqrt_ratio_from_amount_0()`

$$priceNew = \frac{liquidity * priceOld}{liquidity + amount0 * priceOld}$$

`get_next_sqrt_ratio_from_amount_1()`

$$priceNew = \frac{priceOld * liquidity + amount1}{liquidity}$$

Core Invariants:

INV 1: get_next_sqrt_ratio_from_amount_0 must always round up

INV 2: get_amount_0/1 must round up when calculating input amount/ down when calculating output amount

INV 3: feeAmount must always be calculated based on the input amount

Issue_01	Insufficient validation of limit price allows for exploiting swap direction
Severity	**High**
Description	The get_delta_amounts function recalculates swap direction based on price comparison, creating a critical vulnerability where the price movement can be opposite to the actual token flow. This allows malicious users to manipulate market prices while executing trades that appear legitimate. The issue occurs when a user passes zero to one as a parameter on the swap; he can still pass a target price higher than the current price on a sale as the get_delta_amounts function will recalculate the zero to one parameter based on whether the target is smaller or bigger than the current price. The get_delta_amounts recalculation creates a disconnect between: - The token flow direction (determined by the user's original zero_for_one parameter) - The price movement direction (determined by the recalculated zero_for_one value) When these are inconsistent, a trade can execute with tokens flowing in one direction while the price moves in the opposite direction - a fundamental market distortion. This vulnerability allows for direct price manipulation: - A malicious user could execute many small "sell" operations with high price limits - Each operation would successfully sell token0 for token1 (as verified by token flows)

	- Yet each operation would paradoxically drive the price UP - This would create artificial price increases while the attacker is selling
Recommendations	Consider implementing strict validation to ensure consistent token flow direction and price movement direction. Inspiration can be taken from the solidity implementation: *if (* *zeroForOne* *? sqrtRatioLimitX96 >= currentSqrtRatioX96 || sqrtRatioLimitX96 < sqrtRatioAX96* *: sqrtRatioLimitX96 <= currentSqrtRatioX96 || sqrtRatioLimitX96 > sqrtRatioMaxX96* *) revert InvalidSqrtRatioLimit();*
Comments / Resolution	Resolved.

Issue_02	get_next_sqrt_ratio_from_amount_0 should return the same price if the amount is 0
Severity	**Medium**
Description	The protocol intends to implement the UniV3 swapping math. However, it deviates in the implementation of get_next_sqrt_ratio_from_amount_0, as the Univ3 implementation returns the current price if the amount is 0, whereas the Token Mill implementation still performs the calculation. The UniV3 implementation directly returns in this case because the price could still change on a swap amount of 0 due to rounding, which doesn't make sense in the case where nothing is swapped. The current implementation allows for abuse, which could result in unintended price changes without any actual swap.
Recommendations	We recommend returning the current price in get_next_sqrt_ratio_from_amount_0 if the amount is zero.
Comments / Resolution	Resolved.

Issue_03	Creator can DOS Pool at any time (even after the swap authority was removed)
Severity	**Medium**
Description	A market creator can always change the saved fee_reserve by calling the update_fee_reserve IX. This will check if the provided account is a valid token account with the correct mint and set the value on the market account. The user will now need to pass this address on each swap, and Anchor will verify that a token account is stored at this address due to the anchor constraints on the Swap context. *#[account(mut, address = market.fee_reserve.unwrap_or(config.creator_fee_pool))]* *pub fee_reserve: Account<'info, TokenAccount>,* However, as the address is a token account owned by the creator, the creator can close the token account at any time, rendering the anchor constraint ineffective and causing every swap to revert. This attack has a monetary incentive: a creator can artificially decrease the total supply of their token through this attack. As the swaps are DOSed, no new users can purchase additional tokens, and the maximum supply will be frozen at the amount purchased so far.
Recommendations	We recommend verifying manually if the fee_reserve is a valid token account, and if this is not the case, transferring the funds to the creator_fee_pool
Comments / Resolution	Resolved.

Issue_04	The fee split between the protocol and the owner prioritizes the owner
Severity	**Low**
Description	In the distribute_fees function, the fee is split between the protocol and the owner. *let protocol_fee = ((u128::from(fee_amount) * u128::from(self.config.protocol_fee_share))* */ MAX_FEE_U128) as u64;* *let creator_fee = fee_amount - protocol_fee;* As the protocol part is rounded down due to the division, it will result in a Lamport value less than intended.
Recommendations	We recommend rounding towards the protocol.
Comments / Resolution	Acknowledged.

Issue_05	Non-existent swap_authority can be removed
Severity	**Low**
Description	The remove_swap_authority IX feature allows the creator to remove the swap authority set for their market. However, the current implementation does not verify if any swap_authority is set or if the value is already None. If the IX is called with the current value already set to None, it will be overwritten, and another event will be emitted.
Recommendations	We recommend checking if the current swap_authority is None and throwing an error.
Comments / Resolution	Resolved.

Issue_06	Missing check for PriceA <= PriceB
Severity	**Low**
Description	The protocol uses the MarketSettings struct to track the settings with which new markets are generated. This struct includes the two prices sqrt_price_a_x96 and sqrt_price_b_x96. When calculating liquidity, we subtract a from b. *mul_div(* *U256::from(sqrt_price_a) * U256::from(sqrt_price_b),* *U256::from(amount_0),* *// @audit - Potential underflow here as b could be smaller than a* *U256::from(sqrt_price_b - sqrt_price_a).saturating_shl(SQRT_PRICE_SHIFT),* *)* As the code never verifies that PriceA <= PriceB, this could lead to an underflow and a wrongly calculated liquidity.
Recommendations	Consider validating these parameters carefully.
Comments / Resolution	Resolved.

Issue_07	Creator can snipe in same TX as market deployment
Severity	**Low**
Description	Currently, there is no cooldown period between market creation and the first swap call, which means a creator can buy tokens in the same transaction as the market creation, granting themselves an unfair advantage.
Recommendations	Consider implementing a cooldown period after market creation.
Comments / Resolution	Acknowledged.

Issue_08	No validations in create_config handler
Severity	**Low**
Description	The create_config and update_config_settings instructions do not validate the passed inputs. There are no checks to ensure these are set to reasonable values.
Recommendations	Consider implementing checks to ensure the set values are within reasonable bounds.
Comments / Resolution	Acknowledged.

Issue_09	The protocol does not support Token2022
Severity	**Informational**
Description	The current implementation can only be used with standard SPL tokens due to using Program<'info, Token>when creating the market.
Recommendations	If support for Token2022 as a quote token / new token is intended, we recommend adapting the program.
Comments / Resolution	Acknowledged.

Issue_10	The Second Phase swap is triggered with all funds already swapped
Severity	**Informational**
Description	In the get_delta_amounts_from_dual_pool function in the swap_manager, two swapping phases occur. The second phase should begin if the first phase is complete and some swapping delta remains. This is currently checked by the following if statement: *if delta_amount != 0 || new_sqrt_price != sqrt_price_limit* However, if the first phase consumes all the swapping delta, this will initiate the second phase. This will do another swap, which will instantly return zero and waste CU.
Recommendations	We recommend adapting the if clause as follows: *if delta_amount != 0 && new_sqrt_price != sqrt_price_limit*
Comments / Resolution	Resolved.

Issue_11	Unchecked math in the codebase
Severity	**Informational**
Description	As Rust/Solana does not implement automatic under/overflow checks like Solidity, normal arithmetic operations can always overflow. Throughout the codebase, multiple operations are performed that do not utilize checked functions to prevent overflows from occurring.
Recommendations	Since UniswapV3 math is desired to underflow and overflow (which is however based on the logic how feeGrowth is being calculated and not really related to the swap math) this issue may be acknowledged. However, we still recommend carefully considering all possible edge-cases when a over/underflow can happen.
Comments / Resolution	Acknowledged.

Issue_12	The system allows for zero-value swaps
Severity	**Informational**
Description	Implementing both the swap and the swap_with_price_limit IX allows for zero value swaps without reverting. This is typically not supported and can result in the emission of unnecessary events.
Recommendations	We recommend returning an error if a swap with a delta of 0 is created
Comments / Resolution	Resolved.

Issue_13	Division by zero in fee calculation when the fee is set to MAX_FEE during specific swap ops
Severity	**Informational**
Description	The get_delta_amounts function in swap_math.rs contains a division operation where the denominator becomes zero when the fee is set to MAX_FEE (1,000,000). This has been confirmed to occur in the negative delta amount path: *fee_amount = u64::try_from(* *(u128::from(amount_in) * u128::from(fee)).div_ceil(MAX_FEE_U128 - u128::from(fee)),* *).map_err(|_| FeeAmountOverflow)?;* When fee equals MAX_FEE, the denominator (MAX_FEE_U128 - u128::from(fee)) becomes zero. The validation function check_fee in input_validation.rs only rejects fees that are greater than MAX_FEE, not equal to it: *pub fn check_fee(fee: u32) -> Result<u32> {* *if fee > MAX_FEE {* *err!(TokenMillV2Error::InvalidFee)* *} else {* *Ok(fee)* *}* *}* This allows setting the fee to exactly MAX_FEE, which leads to the division by zero when executing swap_with_price_limit with a negative delta amount, as demonstrated in the PoC. While not all paths have been considered vulnerable, they may suffer from the same issue under specific conditions.
Recommendations	We recommend modifying the check_fee function to reject fees that are greater than or equal to MAX_FEE:

	``pub fn check_fee(fee: u32) -> Result<u32> {
	  if fee >= MAX_FEE { // Changed from '>' to '>='
	  err!(TokenMillV2Error::InvalidFee)
	  } else {
	  Ok(fee)
	  }
	}```

Comments / Resolution	Resolved.

Issue_14	Inconsistent fee calculation logic leads to premature routing to the second pool
Severity	**Informational**
Description	The get_delta_amounts function calculates different fee amounts based on which branch of code is executed, leading to unexpected behavior in dual pool swaps. When processing a swap with a calculated target price, a portion of the input amount is unnecessarily routed to the second pool, even though the first pool has not been fully utilized. This occurs due to inconsistent fee calculation methods between the two conditional branches in the get_delta_amounts function. This happens when the price limit will compute exactly the amount_available_in. Testing reveals this issue appears with different parameter combinations, producing varying size discrepancies: 1. Using an exact target price with 101 tokens input and 1% fee: - Processed in first pool: 99 tokens principal + 1 token fee = 100 tokens - Remainder sent to second pool: 1 token 2. Using a much larger input amount (99,999,999,999) with ~66.67% fee: - Results in 2 tokens being unnecessarily routed to the second pool However, when using a slightly higher target price in both cases, all tokens are processed in the first pool with appropriate fees (which is expected and intended). For the second test case (also first one), the 2 tokens routed to the second pool would be entirely consumed as fees, with no actual liquidity change. This means there's no price impact in practice. However, with different parameter values or edge cases not yet

	discovered, this inconsistency could potentially lead to more significant or unexpected price impacts.
Recommendations	This issue is based on the fee calculation logic whether a range can absorb a swap or not which is directly forked from UniswapV3. Therefore, this issue can be acknowledged. However, if this would be considered as a risk, the solution would be to standardize the fee calculation logic in the get_delta_amounts function to ensure consistent behavior regardless of which branch is executed. The fee calculation should use the same formula in both code paths to prevent unintended routing differences.
Comments / Resolution	Acknowledged.